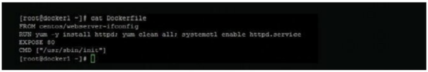
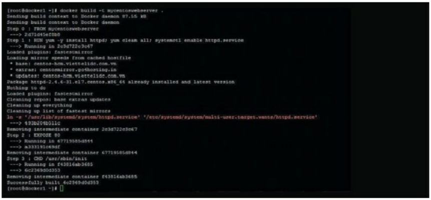


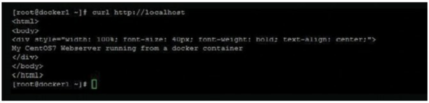
```
[root@docker1 ~]# cat Dockerfile
FROM centos/webserver-ifconfig
RUN yum -y install httpd; yum clean all; systemctl enable httpd.service
EXPOSE 80
CMD ["/usr/sbin/init"]
[root@docker1 ~]# 
```

Figure 7: Sample Dockerfile



```
[root@docker1 ~]# docker build -t mycentoswebserver .
Sending build context to Docker daemon 87.55 kB
Sending build context to Docker daemon
Step 0 : FROM mycentoswebserver
 ---> 2d71d45ef8b8
Step 1 : RUN yum -y install httpd; yum clean all; systemctl enable httpd.service
 ---> Running in 2c3d722c3c67
Loaded plugins: fastestmirror
Loading mirror speeds from cached hostfile
 * base: centos-hcm.viettelidc.com.vn
 * extras: centosmirror.go4hosting.in
 * updates: centos-hcm.viettelidc.com.vn
Package httpd-2.4.6-31.el7.centos.x86_64 already installed and latest version
Nothing to do
Loaded plugins: fastestmirror
Cleaning repos: base extras updates
Cleaning up everything
Cleaning up list of fastest mirrors
ln -s '/usr/lib/systemd/system/httpd.service' '/etc/systemd/system/multi-user.target.wants/httpd.service'
 ---> 493b204b511c
Removing intermediate container 2c3d722c3c67
Step 2 : EXPOSE 80
 ---> Running in 67719585d844
 ---> a333191c69df
Removing intermediate container 67719585d844
Step 3 : CMD /usr/sbin/init
 ---> Running in f43816ab3685
 ---> 6c2369d0d353
Removing intermediate container f43816ab3685
Successfully built 6c2369d0d353
[root@docker1 ~]# 
```

Figure 8: Build using Dockerfile



```
[root@docker1 ~]# curl http://localhost
<html>
<body>
<div style="width: 100%; font-size: 40px; font-weight: bold; text-align: center;">
My CentOS7 Webserver running from a docker container
</div>
</body>
</html>
[root@docker1 ~]# 
```

Figure 9: Testing Docker with a sample index page residing in Web server

```
http_proxy=$HTTP_PROXY
HTTPS_PROXY=$HTTP_PROXY
https_proxy=$HTTP_PROXY
export HTTP_PROXY HTTPS_PROXY http_
proxy https_proxy
```

A sample list of images downloaded to my machine is given in Figure 3.

Docker is now successfully configured and set up in your system, where the machine runs behind a proxy.

Configuring a Web server in the CentOS7 Docker container

This section details how to set up a simple Web server in the Docker container based on CentOS7.

Step 1: To begin, run the command:

```
~# docker run -it docker.io/centos /bin/
bash
```

Note 1: This takes the user to another shell, which is the Docker image terminal.

Note 2: Replace the *docker.io/centos* string in the above command with the downloaded image name as listed in the *docker images* command.

Step 2: In the Centos Docker container terminal, set up *yum.conf* with the proxy details as we did for the host in Step 1 in the section *Setting up Docker in CentOS7, which runs behind a proxy.*

Step 3: Once */etc/yum.conf* is saved with the proxy details, make sure that the container can talk to the outside world via proxy by performing a *yum list* command.

Step 4: To come out of the container without killing it, perform *CTRL-q* followed by *CTRL-P*. If you perform an *exit* command from the container, it comes out of the container and kills the container process. The intention of not killing the instance is to save the Yum setting changes that we performed inside the container.

Step 5: Save the container by executing the following command:

```
a. ~# docker ps
```

```
b. ~# docker commit << ID listed in docker ps command>>
<<Custom Name>>
```

```
c. ~# docker images
```

Step 6: Create a Docker config file to set up a Web server as follows. Create a custom Docker file named *Dockerfile*. The content of the Docker file is shown in Figure 7.

Step 7: Build the Docker container to run the Web server by using the command *docker build –t <<Custom Name>>*

Step 8: Create a test page in the Web server to test it. The default DocumentRoot is */var/www/html/;* hence create a test page *index.html* inside this folder.

Step 9: Once Step 7 is successful, run the command *docker run-privileged-ti-v/sys/fs/cgroup:/sys/fs/cgroup:ro-p 80:80 mycentoswebserver.*

This command is to run the *httpd* daemon inside the container in privileged mode so that *systemd* is started for *httpd*. Without this step, the privilege is not assigned for the container and the *systemd* command will not run on an unprivileged container.

The command run in Step 9 will not be terminated and should not be stopped.

Now it's time to test the docker environment that we created. Invoke the index page that we created in the Web server created inside the docker container as shown in Figure 9.

This article has been written with the purpose of clarifying various doubts about setting up Docker when it is operated behind a proxy. We do hope it has served its purpose. Do let us know! END

By: Krishnaprasad K and Avinash Bendigeri

Krishnaprasad is a software engineer at Dell Inc. His interest areas include automation, virtualisation and cloud computing. He can be reached at *krishnaprasad_k@dell.com.*

Avinash is a software engineer at Dell Inc. His areas of interest include automation and big data analytics.